

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет прикладної математики та інформатики
Кафедра математичного моделювання соціально-економічних процесів

Звіт

Лабораторна робота №4

Виконав:

студент групи Пма-42

Думанський Юрій

Перевірила:

доцент Лисецька О. Ю.

Львів-2026

Тема:

Робота з API

Завдання:

1. Написати скрипт, який завантажує HTML-код веб-сторінки, знаходить усі посилання через регулярні вирази та зберігає їх у файл links.txt.
2. Визначити найбільш доцільний архітектурний стиль API для системи онлайн-бронювання в режимі реального часу.
3. Обрати та пояснити найкращий алгоритм обмеження швидкості запитів до API для запобігання перевантаженню.

Хід роботи:

Для виконання першого завдання було обрано мову програмування Python. За допомогою стандартної бібліотеки urllib реалізовано відправку GET-запиту до сервера для отримання вмісту веб-сторінки у форматі HTML. Для витягування посилань використано модуль re (регулярні вирази). Написано шаблон для пошуку значень атрибута href у тегах <a>. Отриманий список посилань збережено у текстовий файл links.txt.

У другому завданні проаналізовано стилі REST, GraphQL та WebSocket. Для системи бронювання, де критично важливо відображати зміни статусів у реальному часі без постійного оновлення сторінки користувачем, обрано протокол WebSocket. Це дозволяє серверу миттєво передавати дані клієнту через відкрите двостороннє з'єднання.

У третьому завданні розглянуто методи захисту API від перевантаження (Rate Limiting). Обрано алгоритм Token Bucket. Основна причина вибору полягає в його здатності ефективно обробляти короткочасні сплески активності (bursts), зберігаючи при цьому задану середню швидкість обробки запитів для конкретного клієнта.

Результати:

Написаний скрипт успішно зчитує структуру HTML-документа та формує перелік URL-адрес у зовнішньому файлі. Проведено теоретичне обґрунтування використання WebSocket для інтерактивних систем та алгоритму Token Bucket для контролю трафіку, що забезпечує стабільну роботу веб-сервісу при високих навантаженнях.

Висновки:

Під час лабораторної роботи було реалізовано автоматизований збір даних з веб-сторінок за допомогою регулярних виразів. Вивчено архітектурні особливості побудови сучасних API та методи їх захисту від надмірної кількості запитів. Отримані результати підтверджують ефективність обраних інструментів для вирішення поставлених задач.

Додаток 1. Код програми.

```
import urllib.request
import urllib.parse
import re

def extract_links(url, output_file="links.txt"):
    try:
        # Відправляємо GET-запит за вказаною URL-адресою
        req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
        response = urllib.request.urlopen(req)

        # Читаємо та декодуємо HTML-код сторінки
        html_content = response.read().decode('utf-8', errors='ignore')

        # Регулярний вираз для пошуку атрибутів href всередині тегів <a>
        # Шукає конструкцію: <a ... href="URL" ...>
        regex = r'<a[^\>]+href=["\'](.*)["\']>'

        # Знаходимо всі збіги
        links = re.findall(regex, html_content)

        # Зберігаємо знайдені посилання у текстовий файл
        with open(output_file, 'w', encoding='utf-8') as file:
            for link in links:
                link=urllib.parse.unquote(link)
                file.write(link + '\n')

        print(f"Успішно знайдено посилань: {len(links)}.")
        print(f"Результати збережено у файл: {output_file}")

    except Exception as e:
        print(f"Сталася помилка під час обробки {url}: {e}")

# Приклад виклику функції
if __name__ == "__main__":
    target_url = "https://uk.wikipedia.org" # Замініть на потрібний URL
    extract_links(target_url)
```